

Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)

(An Autonomous Institute)

Affiliated to Savitribai Phule Pune University(SPPU)

ISO 21001:2018 Certified by TUV SUD

TY B.Tech

Year:2025-26

Semester:6

Name: Paritosh Nimhan

PRN:123B1B212

Branch: Computer Engineering

Div:C

Assignment 6

Title: Create an online appointment booking application/ E-commerce portal for used items sales like car, bike etc. or a similar type application using Node.js and Express, using any NoSQL dataset like MongoDB if required.

Objectives:

- To understand the fundamentals of backend web development using Node.js and Express.js.
- To design and implement a real-world application such as an online appointment booking system or used-items e-commerce platform.

Outcomes:

- Developed a web-based application using Node.js and Express.js.
- Implemented RESTful APIs for managing users, products/appointments, and bookings.
- Integrated MongoDB (NoSQL database) for storing application data.

Theory Concept:

Introduction

With the rapid growth of digital platforms, online systems such as appointment booking applications and e-commerce portals have become essential. These systems help users perform tasks such as scheduling appointments or buying/selling used items like cars and bikes conveniently from anywhere.

In this practical, we develop a backend application using Node.js and Express, along with MongoDB as the database. The application supports features like user authentication, listing items, and booking or purchasing services.

Node.js

Node.js is an open-source, cross-platform runtime environment that allows developers to execute JavaScript on the server side. It is built on Chrome's V8 engine and is known for its non-blocking, event-driven architecture.

Key Features:

- Asynchronous and event-driven
- Fast execution
- Scalable for real-time applications
- Uses JavaScript for both frontend and backend

Express.js

Express.js is a lightweight web framework for Node.js used to build web applications and APIs.

Key Features:

- Simplifies routing and middleware handling
- Supports REST API development
- Easy integration with databases
- Flexible and minimalistic

MongoDB (NoSQL Database)

MongoDB is a NoSQL database that stores data in JSON-like documents (BSON format). It is schema-less, making it flexible for applications where data structure may change.

Advantages:

- High performance
- Flexible schema
- Easy scalability
- Suitable for large datasets

Application Overview (Car Service Booking System)

The application is an **online car service appointment booking system**.

Users can:

- Register/Login (optional based on backend)
- Fill booking form (name, car model, service type, date & time)
- Book car service appointments
- Search their bookings using name
- View or cancel appointments

Admin/System can:

- Manage appointment data
- Store bookings in database
- Handle booking requests via API
- Allow search and cancellation of bookings

System Architecture

The application follows a **client-server architecture**:

- Frontend (optional): HTML, CSS, JavaScript
- Backend: Node.js + Express

- Database: MongoDB

Dataset Used

- No external dataset was used.
- Data is dynamically generated and stored in MongoDB collections during runtime

Code:

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Car Service Booking</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>

  <div class="container">
    <div style="text-align: center;">
      
    </div>
    <h1>Book a Car Service Appointment</h1>

    <p>Please fill out the form below to book your appointment.</p>

    <form id="bookingForm">
      <div class="form-group">
        <label for="customerName">Full Name:</label>
        <input type="text" id="customerName" name="customerName" required>
      </div>

      <div class="form-group">
        <label for="carModel">Car Model (e.g., Honda Civic 2018):</label>
        <input type="text" id="carModel" name="carModel" required>
      </div>

      <div class="form-group">
        <label for="serviceType">Type of Service:</label>
        <select id="serviceType" name="serviceType" required>
          <option value="">--Select a Service--</option>
          <option value="General Servicing">General Servicing</option>
          <option value="Oil Change">Oil Change</option>
          <option value="Brake Inspection">Brake Inspection</option>
          <option value="Engine Diagnostics">Engine Diagnostics</option>
          <option value="Other">Other</option>
        </select>
      </div>

      <div class="form-group">
```

```

        <label for="preferredDate">Preferred Date:</label>
        <input type="date" id="preferredDate" name="preferredDate" required>
    </div>

    <div class="form-group">
        <label for="preferredTime">Preferred Time:</label>
        <input type="time" id="preferredTime" name="preferredTime" required>
    </div>

    <button type="submit">Book Appointment</button>
</form>

    <div id="message" style="display:none; margin-top: 15px; padding: 10px; border: 1px solid green;
background-color: #d4edda; color: #155724;"></div>
    <div id="error-message" style="display:none; margin-top: 15px; padding: 10px; border: 1px solid
red; background-color: #f8d7da; color: #721c24;"></div>

</div>

<!-- Section to manage appointments -->
<div class="container" style="margin-top: 30px;">
    <h2>Manage Your Booking</h2>
    <p>Search by your full name to view or cancel your upcoming appointments.</p>

    <form id="searchForm">
        <div class="form-group">
            <input type="text" id="searchName" placeholder="Enter your full name" required>
        </div>
        <button type="submit">Search Booking</button>
    </form>

    <div id="searchMessage" style="display:none; margin-top: 15px; padding: 10px; border: 1px solid
green; background-color: #d4edda; color: #155724;"></div>
    <div id="searchError" style="display:none; margin-top: 15px; padding: 10px; border: 1px solid red;
background-color: #f8d7da; color: #721c24;"></div>

    <ul id="searchResults" style="margin-top: 20px; list-style-type: none; padding-left: 0;">
        <!-- Results will be displayed here -->
    </ul>
</div>

<script src="script.js"></script>
</body>
</html>

```

script.js

```

document.getElementById('bookingForm').addEventListener('submit', function(event) {
    event.preventDefault(); // Prevent default form submission

    // Hide messages

```

```

document.getElementById('message').style.display = 'none';
document.getElementById('error-message').style.display = 'none';

// Get form data
const formData = {
  customerName: document.getElementById('customerName').value,
  carModel: document.getElementById('carModel').value,
  serviceType: document.getElementById('serviceType').value,
  preferredDate: document.getElementById('preferredDate').value,
  preferredTime: document.getElementById('preferredTime').value
};

// Send POST request
fetch('/api/appointments', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(formData)
})
.then(async response => {
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.error || 'Network response was not ok');
  }
  return data;
})
.then(data => {
  // Show success message
  const msgDiv = document.getElementById('message');
  msgDiv.innerText = data.message;
  msgDiv.style.display = 'block';

  // Reset form
  document.getElementById('bookingForm').reset();
})
.catch(error => {
  // Show error message
  const errDiv = document.getElementById('error-message');
  errDiv.innerText = error.message || "Error booking appointment. Please try again later.";
  errDiv.style.display = 'block';
  console.error('There was a problem with the fetch operation:', error);
});

document.getElementById('searchForm').addEventListener('submit', function(event) {
  event.preventDefault();
  const name = document.getElementById('searchName').value.trim();

  document.getElementById('searchMessage').style.display = 'none';
  document.getElementById('searchError').style.display = 'none';
});

```

```

fetch(`/api/appointments/search?name=${encodeURIComponent(name)}`)
  .then(res => res.json())
  .then(data => {
    const list = document.getElementById('searchResults');
    list.innerHTML = "";

    if (data.length === 0) {
      list.innerHTML = '<li>No appointments found for this name.</li>';
      return;
    }

    data.forEach(appt => {
      const li = document.createElement('li');
      li.style.marginBottom = '20px';
      li.style.padding = '15px';
      li.style.border = '1px solid #ddd';
      li.style.borderRadius = '5px';
      li.style.backgroundColor = '#f9f9f9';

      const dateObj = new Date(appt.preferredDate);
      const dateStr = dateObj.toLocaleDateString();

      li.innerHTML = `<strong>${appt.carModel}</strong> - ${appt.serviceType} <br> 📅 Date:
      ${dateStr} <br> 🕒 Time: ${appt.preferredTime} <br>`;

      const cancelBtn = document.createElement('button');
      cancelBtn.innerText = 'Cancel Appointment';
      cancelBtn.style.marginTop = '10px';
      cancelBtn.style.backgroundColor = '#dc3545';
      cancelBtn.style.color = '#fff';
      cancelBtn.style.border = 'none';
      cancelBtn.style.padding = '8px 12px';
      cancelBtn.style.cursor = 'pointer';
      cancelBtn.style.borderRadius = '4px';

      cancelBtn.addEventListener('click', () => cancelAppointment(appt._id, name));

      li.appendChild(cancelBtn);
      list.appendChild(li);
    });
  })
  .catch(err => {
    const errDiv = document.getElementById('searchError');
    errDiv.innerText = 'Failed to search appointments.';
    errDiv.style.display = 'block';
  });
});

function cancelAppointment(id, searchName) {
  if (!confirm('Are you sure you want to cancel this appointment?')) return;

  fetch(`/api/appointments/${id}`, { method: 'DELETE' })

```

```

.then(res => res.json())
.then(data => {
  const msgDiv = document.getElementById('searchMessage');
  if (data.error) {
    const errDiv = document.getElementById('searchError');
    errDiv.innerText = data.error;
    errDiv.style.display = 'block';
  } else {
    msgDiv.innerText = data.message || 'Appointment cancelled!';
    msgDiv.style.display = 'block';
    // Trigger search again to refresh the list
    document.getElementById('searchForm').dispatchEvent(new Event('submit'));
  }
})
.catch(err => {
  const errDiv = document.getElementById('searchError');
  errDiv.innerText = 'Failed to cancel appointment.';
  errDiv.style.display = 'block';
});
}

```

Server.js

```

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const Appointment = require('./models/Appointment');

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// Connect to MongoDB Atlas
mongoose.connect(process.env.MONGODB_URI)
.then(() => console.log('Connected to MongoDB Atlas'))
.catch(err => console.error('Could not connect to MongoDB:', err));

// Routes
app.post('/api/appointments', async (req, res) => {
  try {
    // Check for existing appointment
    const existingAppointment = await Appointment.findOne({
      preferredDate: req.body.preferredDate,
      preferredTime: req.body.preferredTime
    });

```

```

});

if (existingAppointment) {
  return res.status(400).json({ error: 'Appointment not available for this date and time. Please choose another slot.' });
}

const newAppointment = new Appointment({
  customerName: req.body.customerName,
  carModel: req.body.carModel,
  serviceType: req.body.serviceType,
  preferredDate: req.body.preferredDate,
  preferredTime: req.body.preferredTime
});

const savedAppointment = await newAppointment.save();
res.status(201).json({ message: 'Appointment booked successfully!', appointment:
savedAppointment });
} catch (error) {
  console.error('Error saving appointment:', error);
  res.status(500).json({ error: 'Failed to book appointment' });
}
});

app.get('/api/appointments', async (req, res) => {
  try {
    const appointments = await Appointment.find().sort({ createdAt: -1 });
    res.json(appointments);
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch appointments' });
  }
});

```



Book a Car Service Appointment

Please fill out the form below to book your appointment.

Full Name:

Car Model (e.g., Honda Civic 2018):

Type of Service:

General Servicing
⌵

Preferred Date:

Preferred Time:

Book Appointment

Manage Your Booking

Search by your full name to view or cancel your upcoming appointments.

Search Booking

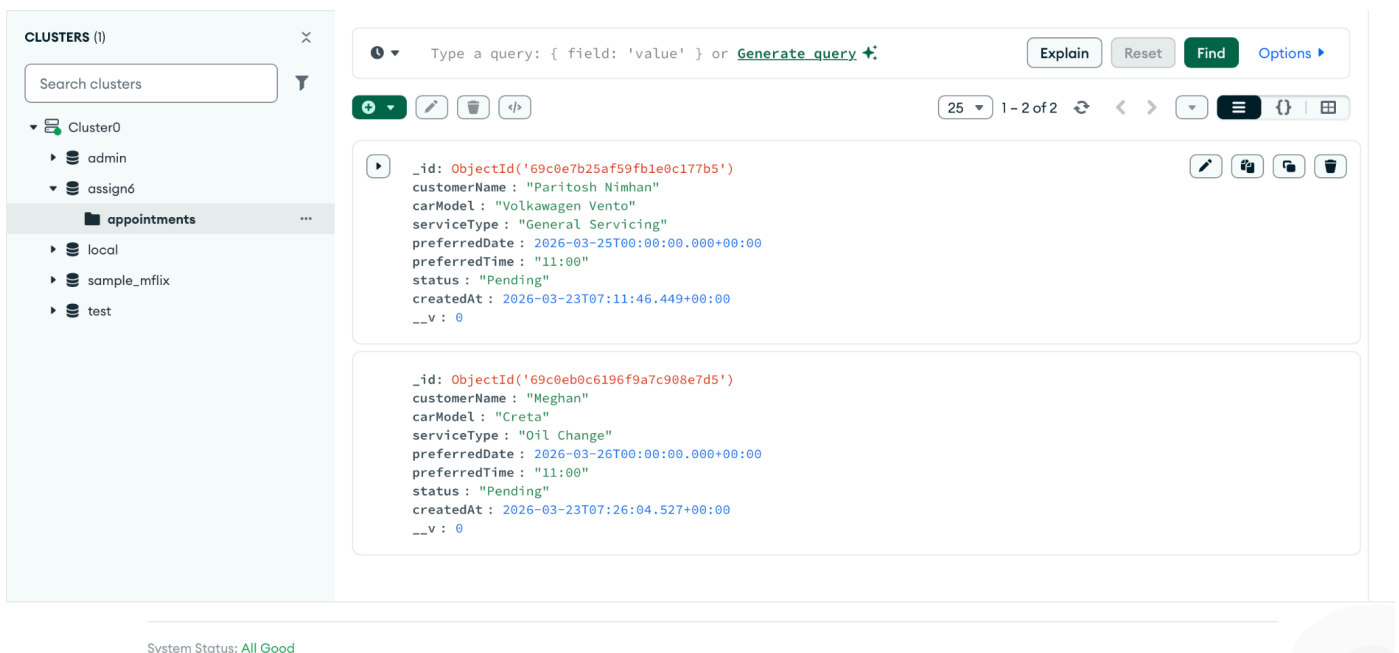
Volkawagen Vento - General Servicing

 Date: 25/3/2026

 Time: 11:00

Cancel Appointment

Connection with MongoDB:



The screenshot displays the MongoDB Compass web interface. On the left, a sidebar shows the 'CLUSTERS (1)' section with a search bar and a tree view containing 'Cluster0', 'admin', 'assign6', 'appointments' (selected), 'local', 'sample_mflix', and 'test'. The main area shows a query bar with the text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar, there are two document entries in a list view. The first document is for 'Paritosh Nimhan' with a car model of 'Volkawagen Vento' and a service type of 'General Servicing'. The second document is for 'Meghan' with a car model of 'Creta' and a service type of 'Oil Change'. Both documents show a 'status' of 'Pending' and a 'createdAt' timestamp. At the bottom left, a status bar indicates 'System Status: All Good'.

```
{
  "_id": ObjectId('69c0e7b25af59fb1e0c177b5'),
  "customerName": "Paritosh Nimhan",
  "carModel": "Volkawagen Vento",
  "serviceType": "General Servicing",
  "preferredDate": "2026-03-25T00:00:00.000+00:00",
  "preferredTime": "11:00",
  "status": "Pending",
  "createdAt": "2026-03-23T07:11:46.449+00:00",
  "__v": 0
}
```

```
{
  "_id": ObjectId('69c0eb0c6196f9a7c908e7d5'),
  "customerName": "Meghan",
  "carModel": "Creta",
  "serviceType": "Oil Change",
  "preferredDate": "2026-03-26T00:00:00.000+00:00",
  "preferredTime": "11:00",
  "status": "Pending",
  "createdAt": "2026-03-23T07:26:04.527+00:00",
  "__v": 0
}
```

Conclusion

In this practical, we successfully developed a backend application using Node.js and Express, integrated with MongoDB. The project demonstrated how modern web applications handle data, user interactions, and business logic efficiently.